

Multi-Agent Research Analyst

Moving from a single-loop AI Agent to an Agentic AI system by orchestrating multiple specialized sub-agents, demonstrating parallel execution, shared state management, and self-correcting feedback loops.

🐍 Python

🤖 LangGraph

⚡ Groq (Llama 3.3 70B)

🔍 Tavily (Web Search)

💻 Streamlit

OBJECTIVE

PROJECT GOAL

To move from a single-loop AI Agent to an Agentic AI system by orchestrating multiple specialized sub-agents using LangGraph, demonstrating parallel execution, shared state management, and self-correcting feedback loops.

FOUNDATION

Core Concepts



Agent

An LLM placed in a loop — LLM with Hands, LLM + Tool + System Prompt. Given a goal, it can **reason and plan**, use tools, observe results, and loop until the given goal is achieved.



Tools

A function or method which takes defined inputs/arguments and returns a result or performs an action. Tools act as **hands** that perform actions — get data, extract data, hit API requests — in external apps or systems.



Graph

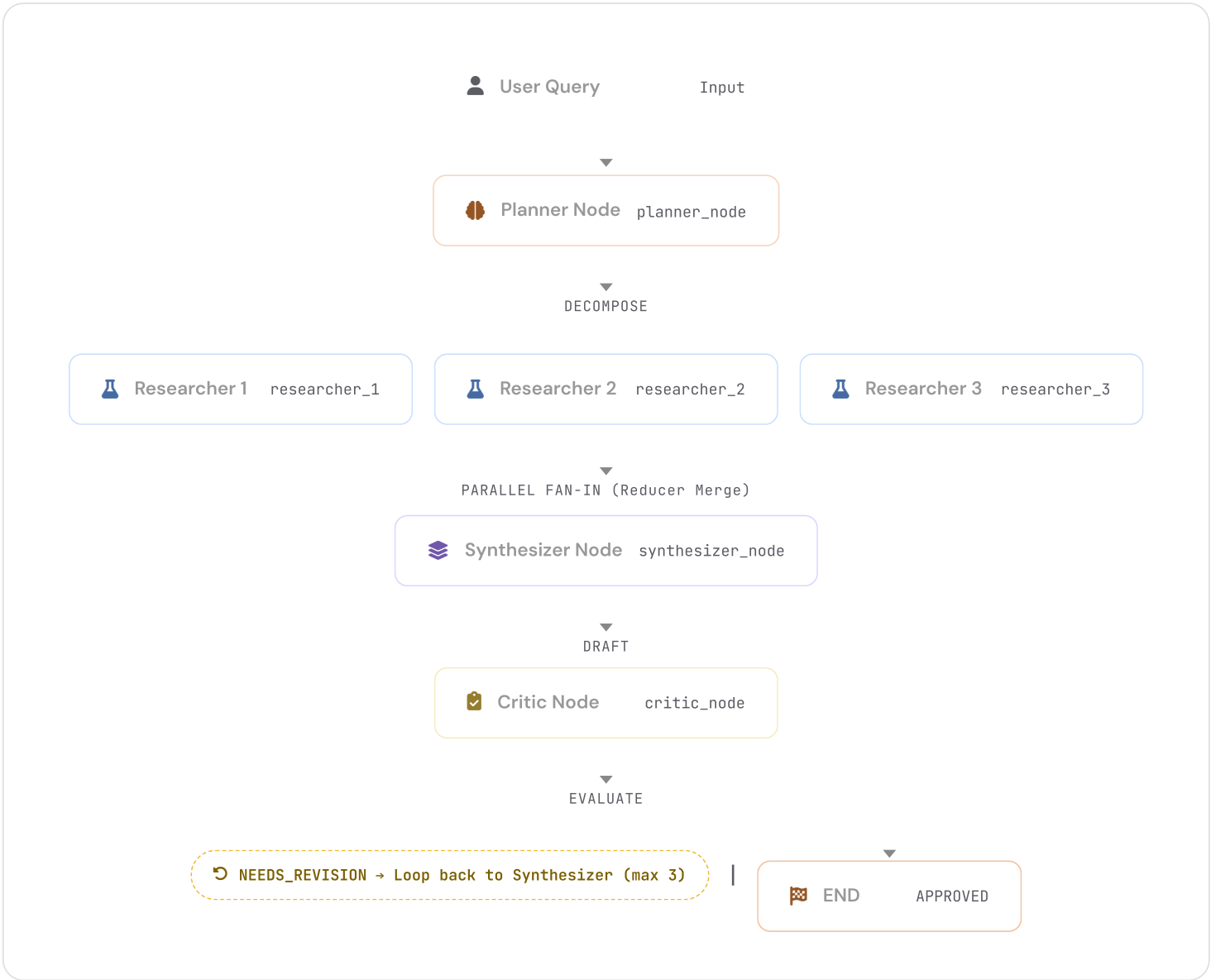
A structure of **connected nodes with edges**. The graph defines the flow of data and control between different units of work, enabling complex orchestration patterns like fan-out, fan-in, and conditional loops.



Nodes

A unit of work in an agent graph. Programmatically, it is a function that receives the **current graph state**, performs an operation (calling an LLM, tool, API, database, or Python logic), and returns updates to the state.

System Architecture



ORCHESTRATION

Multi-Agent Pattern Mapping

The system explicitly realizes the three foundational Agentic AI patterns, each mapped directly to specific nodes and edges in the LangGraph implementation.

COURSE PATTERN	IMPLEMENTATION	HOW IT WAS REALIZED
Planner-Executor Delegation	<code>planner_node</code> → <code>researcher_1,2,3</code>	The Planner LLM decomposes the complex question into 3–5 discrete sub-tasks. It does not execute the research; it only creates the execution plan for the workers.
Orchestrator-Workers Parallelism	<code>graph.py</code> Fan-out / Fan-in	The graph draws 3 edges from the Planner to <code>researcher_1,2,3</code> . LangGraph executes these in parallel threads. They safely merge their findings using the orchestrator's reducer on the <code>findings</code> state key before feeding into the Synthesizer.
Evaluator-Optimizer Self-Correction	<code>critic_node</code> → <code>synthesizer_node</code>	A conditional edge routes the Critic's output. If the draft is <code>APPROVED</code> , it goes to <code>END</code> . If <code>NEEDS_REVISION</code> , it loops back to the

COURSE PATTERN	IMPLEMENTATION	HOW IT WAS REALIZED
		Synthesizer. An iteration cap (<code>MAX_ITERATIONS = 3</code>) prevents infinite loops.

BENCHMARKING

Evaluation & Baseline Comparison

To prove the value of orchestration, the system was benchmarked against a Part-1 **Single-Agent Baseline** (a single ReAct loop with web search). An LLM-as-Judge scored both outputs 1–5 on Coverage, Faithfulness, Citation Quality, and Coherence.

4.75

Multi-Agent Score
out of 5.00

2.75

Baseline Score
out of 5.00

6.61s

Multi-Agent Latency
vs 7.37s baseline



Winner: Multi-Agent System

72.7% score improvement over single-agent baseline, with comparable latency due to parallel execution.

LLM-AS-JUDGE RAW OUTPUT

```
[
  {
    "topic": "Compare the current EV strategies of Tata Mot...",
    "baseline_score": 2.75,
    "multiagent_score": 4.75,
    "baseline_time": 7.37,
    "multiagent_time": 6.61,
    "winner": "Multi-Agent"
  }
]
```



Cost & Latency Trade-off

Agentic AI trades higher cost and latency for higher reliability. While the single-agent baseline is fast and cheap, it frequently produces shallow answers or hallucinates citations. The multi-agent system consumes significantly more Groq tokens (due to multiple planner/researcher/critic calls) and takes longer in theory, but yields structured, multi-perspective briefs with verified citations. In this benchmark, parallel execution actually made the multi-agent system slightly faster than the sequential baseline.



Infrastructure Note

The multi-agent system also experienced rate-limiting (HTTP 429) on the Groq free tier during parallel researcher execution, highlighting the infrastructural requirements of Agentic systems in production deployments.

REFLECTIONS

New Perspectives from Building This Project



01

Usage Cost Per Execution

Every LLM call in the agent loop has a real cost. With multiple sub-agents (Planner, 3 Researchers, Synthesizer, Critic), token consumption multiplies rapidly. Tracking cost per execution is essential for production viability.



02

Latency & Response Speed

User experience demands fast responses. Targeting sub-10ms perceived latency requires careful orchestration — parallel execution, streaming outputs, and minimizing sequential LLM calls wherever possible.



03

Security & Guardrails

Agents that can call tools and APIs need robust input/output guardrails. Prompt injection, unauthorized tool access, and leaking sensitive data in LLM responses are real attack surfaces that must be hardened.



04

Evaluation of Agent Trajectories

Final output quality alone is insufficient. Evaluating the full agent trajectory — which tools were called, why the critic looped, whether the planner decomposed correctly — is critical for debugging and improving agent behavior.